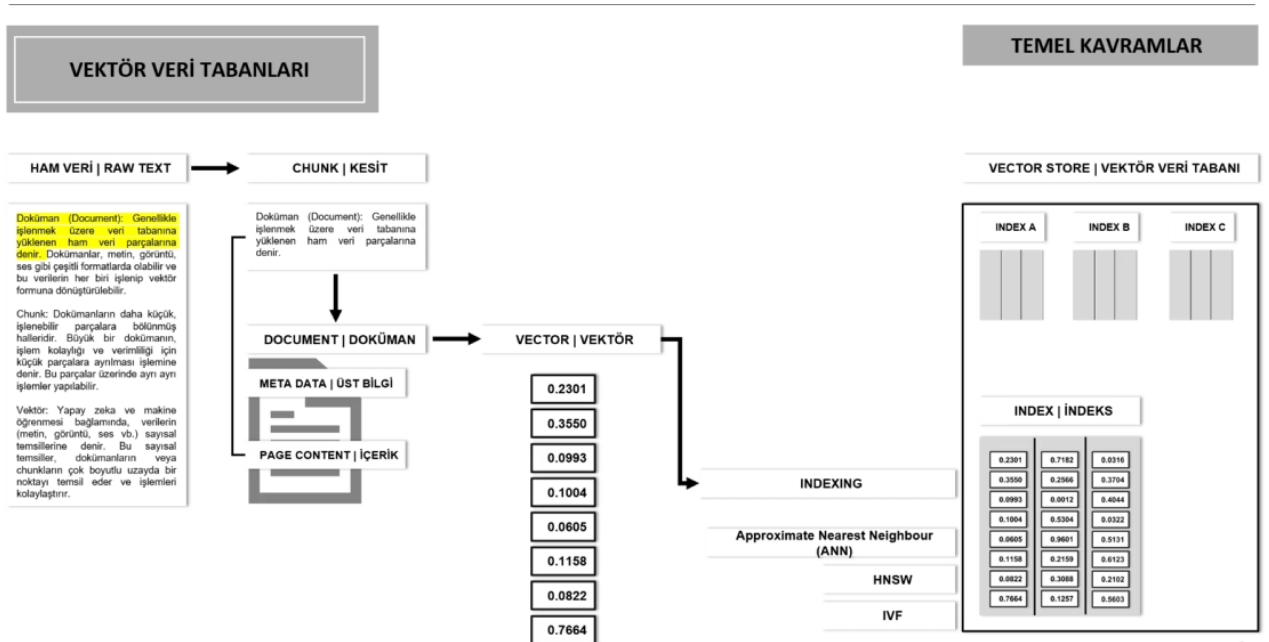


Bellek genişletme (RAG) stratejilerinde verinin matematiksel yolculuğunu (embedding) öğrendikten sonra, şimdi bu verilerin nasıl depolandığını, yönetildiğini ve en önemlisi nasıl verimli bir şekilde geri çağrıldığını inceleyeceğiz. Vektör Veri Tabanları, klasik veri tabanlarından farklı olarak yüksek boyutlu sayı dizilerini (vektörleri) saklamak ve bunlar üzerinde anlamsal aramalar yapmak üzere özelleşmiş sistemlerdir.

1. Veri Odaklı Bakış: Ham Veriden Vektör Mağazasına (Vector Store)

Bir verinin vektör veri tabanına girmeden önceki hazırlık süreci, arama isabetliliğini doğrudan etkiler.



Bölümlleme (Chunking): Ham veri küçük parçalara ayrılarak yönetilebilir "kesitler" oluşturulur.

Döküman (Document) Sınıfı: Kesitler, LangChain gibi çerçevelerde standart bir döküman yapısına dönüştürülür. Bu yapı iki ana bileşenden oluşur:

Metadata (Üst Bilgi): Verinin kaynağı, sayfa numarası veya saniyesi gibi "nereden geldiğini" belirten etiketlerdir. İyi tasarlanmış bir metadata, retrieval (bulup getirme) sonrası bilginin doğruluğunu anlamak için kritiktir.

Page Content (İçerik): Asıl bilgi bölümüdür.

Vektörizasyon: Döküman içeriği embedding modelleriyle ondalıklı sayı dizilerine dönüştürülür ve ardından indeksleme aşamasına geçilir.

2. İndeksleme Stratejileri ve ANN Algoritmaları

Vektör veri tabanlarında binlerce veya milyonlarca vektör arasından arama yaparken her birini tek tek kıyaslamak (brute force / linear scan) son derece yavaştır. Bunun yerine, Approximate Nearest Neighbour (ANN - Yaklaşık En Yakın Komşu) algoritmaları kullanılır.

İndeksleme aşamasında vektörler gelişi güzel değil, benzerliklerine göre gruplandırılarak yerleştirilir. Bu süreçte iki yaygın algoritma öne çıkar:

HNSW (Hierarchical Navigable Small World): Arama hızını maksimize eder ancak indeksleme süresi daha uzundur. Bir kez indeks oluştuktan sonra sonuçlar çok hızlı döner.

IVF (Inverted File Index): İndeksleme aşaması çok hızlıdır ancak arama performansı HNSW'ye göre daha yavaştır.

Arama sonucunda bulunan adaylar Re-ranking (yeniden sıralama) işlemine tabi tutularak kullanıcıya en ilgili "k" adet döküman (genellikle varsayılan 4) döndürülür.

3. Uygulama ve Depolama Arasındaki Köprü: Retriever

Vektör veri tabanı ile uygulama arasındaki etkileşimi sağlayan sınıflara Retriever denir. Retriever'lar şu kritik görevleri üstlenir:

Vektör karşılaştırma talimatını vermek.

Vektör veri tabanına yazma ve okuma işlemlerini yönetmek.

Eşleşen vektöre karşılık gelen orijinal dökümanı bulup getirmek.

4. Vektör Veri Tabanı Alternatifleri

Günümüzde vektör depolama ihtiyacı için iki ana yaklaşım mevcuttur:



Özelleşmiş Vektör Veri Tabanları

Pinecone: Profesyonel ve yüksek performanslı, ücretli bir bulut servsidir.

Chroma: Açık kaynaklı, geliştirici dostu ve optimizasyon araçları sunan güçlü bir alternatiftir.

Faiss: Facebook (Meta) tarafından geliştirilen, kalıcı depolamadan ziyade in-memory (bellek içi) hızlı arama işlemlerine odaklanan bir kütüphanedir.

Diğerleri: Qdrant, Milvus ve Weaviate gibi açık kaynaklı başarılı girişimler de mevcuttur.

Genel Amaçlı Veri Tabanları (Vektör Destekli)

Geleneksel yazılım dünyasından tanıdığımız PostgreSQL (pgvector), MongoDB ve Elasticsearch gibi servisler de vektör saklama ve indeksleme yetenekleri ekleyerek bu alanda yerlerini almışlardır. Eğer bu teknolojilere zaten aşinaysanız, projenize buradan başlamak mantıklı bir adım olabilir.